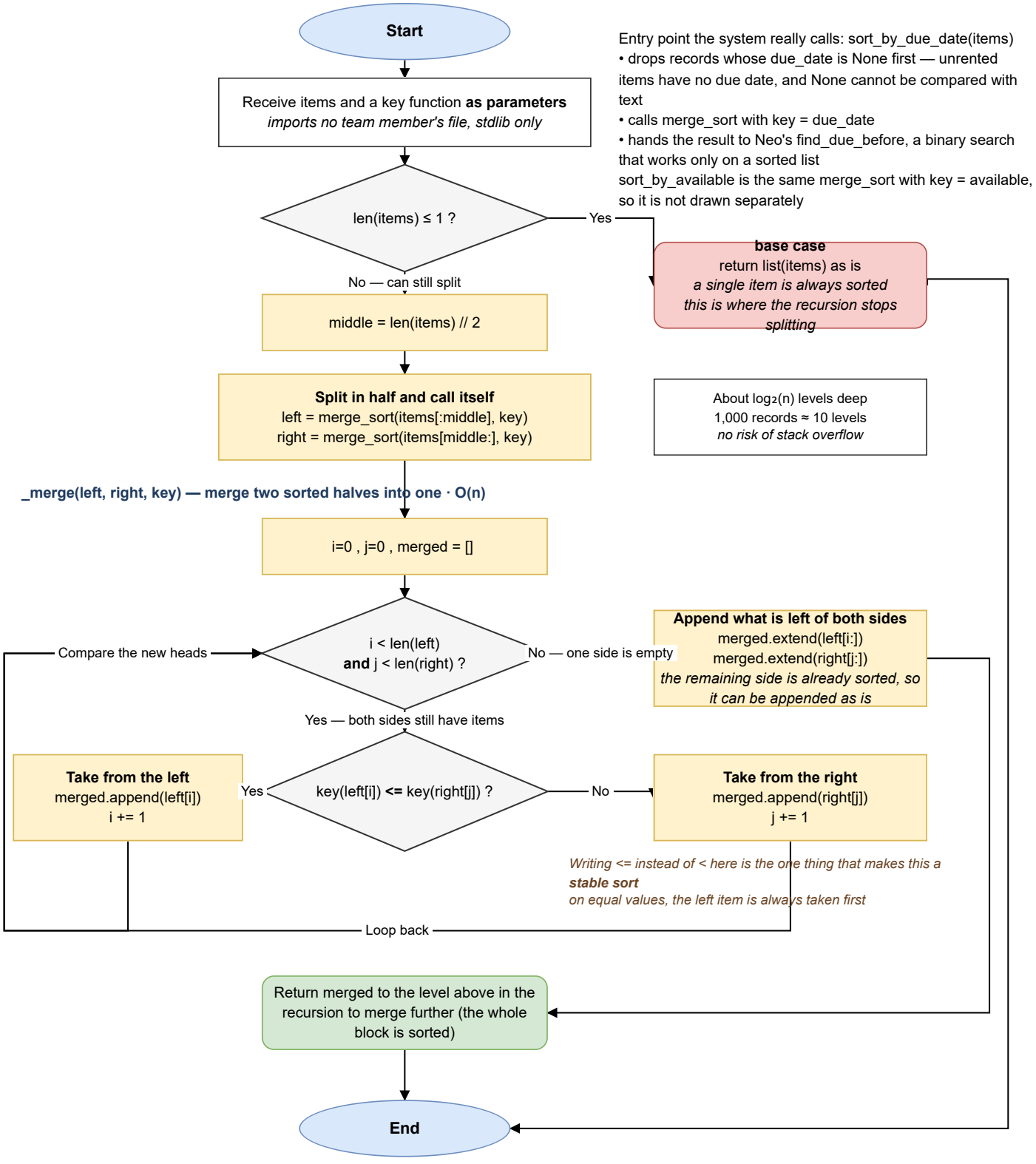
# Flowchart 6 — Sorting: merge_sort (split in half → merge)

sort_group.py · built as recursion that splits in half until one item is left, unlike hash_table.py, which computes the position straight from the key, and unlike count_* in search.py, which walks the data once




Entry point the system really calls: sort_by_due_date(items)
• drops records whose due_date is None first — unrented items have no due date, and None cannot be compared with text
• calls merge_sort with key = due_date
• hands the result to Neo's find_due_before, a binary search that works only on a sorted list
sort_by_available is the same merge_sort with key = available, so it is not drawn separately

**Why splitting in half and merging is faster than swapping pair by pair**

• Splitting in half until one item is left gives **log₂(n) levels**, and merging each level costs **O(n)**, for **O(n log n)** in total
• Bubble sort compares every pair over and over, so it is **O(n²)** — measured on set D with 1,000 records, **merge sort is 34.6 times faster than bubble sort**
• Writing **<= instead of <** when comparing the left side makes it a **stable sort**, so records with the same due_date keep their original order instead of being swapped for no reason
• It **always returns a new list and never changes the original**, because someone else may hold the same list, just like sort_by_code in search.py
• The sorted list from this file is what **find_due_before (Neo's binary search)** uses directly; if it were sorted wrongly, the halving would cut the wrong side **with no error at all**